

Day 1 — 语言基础速通（完整版）

你有多多年编程经验且接触过 Python，不讲"变量是什么"。直接按 Python 和其他语言差异最大的点来，快速建立完整的 Python 心智模型。

一、Python 的"哲学"——先理解为什么

1. 是什么

打开 Python 终端执行 `import this`，会看到 Tim Peters 写的《Python 之禅》——这是 Python 语言设计的核心指导思想。

```
Beautiful is better than ugly.  
Explicit is better than implicit.  
Simple is better than complex.  
Complex is better than complicated.  
Readability counts.
```

2. 解决了什么问题

Python 的哲学解决了一个元问题：当语言设计遇到选择时，该往哪个方向走？

对比一下：

```
# 不 Pythonic (像 Java 翻译过来)  
def calculate_value(a, b, c):  
    list_object = new_list()  
    list_object.append(a)  
    return_value = 0  
    for i in list_object:  
        return_value = return_value + i
```

```
    return return_value

# Pythonic
def calculate(a, b, c):
    return sum([a, b, c])
```

Python 的哲学选择"显式优于隐式"、"可读性优先"。这意味着： - API 设计通常用清晰的名称，而不是缩写 - 没有花括号——缩进本身就是代码块 - 不搞隐式类型转换——宁可报错也不猜

3. 核心理论

Python 的三个核心设计决策贯穿了所有语法：

决策 **1**：动态强类型

```
# 动态：变量不需要声明类型
x = 42
x = "hello" # 合法

# 强：不会隐式类型转换
"answer: " + 42 # ✗ TypeError
"answer: " + str(42) # ✓
```

跟 JS（弱类型）对比：

```
"5" + 3 = "53" // JS 自动转
"5" - 3 = 2 // JS 自动转
```

Python 不会做这种事。

决策 **2**：缩进即块结构

```
for i in range(3):
    if i % 2 == 0:
        print(f"{i} is even")
    else:
```

```
    print(f"{i} is odd")
print("done") # 在 for 外面
```

标准约定：**4** 个空格，永远不要混用 Tab。

决策 **3**：一切皆对象

```
def hello():
    return "hello"

hello.custom_attr = 42 # 函数可以动态加属性
print(hello.__name__) # "hello"

# 函数可以传给变量、作为返回值
def call_twice(func):
    return func() + func()

print(call_twice(hello)) # "hellohello"
```

对"一切皆对象"的深入理解，是打通 Python 高级主题（装饰器、元类、描述器）的关键。

二、核心数据类型

1. 是什么

Python 的内置数据类型比大多数语言丰富，而且每个都有独特的设计。

类型	可否变	可哈希	一句话
<code>int</code>	—	✓	任意精度，不会溢出
<code>float</code>	—	✓	底层 C double，有精度问题
<code>complex</code>	—	✓	原生复数

类型	可否变	可哈希	一句话
<code>bool</code>	—	✓	int 子类，True=1
<code>str</code>	✗	✓	Unicode，不可变
<code>list</code>	✓	✗	最常用容器
<code>tuple</code>	✗	✓	不可变列表
<code>set</code>	✓	✗	无序不重复
<code>dict</code>	✓	✗	关联数组
<code>NoneType</code>	—	—	有且只有 None

2. 解决了什么问题

每个类型解决了一个具体的痛点：

- **int** 任意精度 — 不用再考虑整数溢出（其他语言的老大难）
- **bool** 是 **int** 子类 — 可以直接做数学运算，隐式布尔转换让条件判断优雅
- **tuple** 可哈希 — 可以作为字典键（list 不行）
- **set O(1)** 查找 — 比 list 快得多

3. 核心理论

int — 任意精度整数

```
# 不会溢出
a = 2 ** 1000 # 300+ 位的大整数
print(a)

# 进制直接量
print(0b1010) # 10（二进制）
print(0o12)   # 10（八进制）
print(0xA)    # 10（十六进制）
print(bin(10)) # "0b1010"
```

```

print(hex(10))    # "0xa"

# 大整数运算
factorial_100 = 1
for i in range(1, 101):
    factorial_100 *= i
print(factorial_100)  # 158 位数字——其他语言早就溢出了

```

float — 底层是 C double

```

# 浮点精度问题（跟所有语言一样，因为 IEEE 754）
print(0.1 + 0.2)          # 0.30000000000000004
print(0.1 + 0.2 == 0.3)   # False ✕

# 正确比较方式
print(abs(0.1 + 0.2 - 0.3) < 1e-9) # True
print(round(0.1 + 0.2, 2) == 0.3)   # True

# Decimal 精确小数（用于金融场景）
from decimal import Decimal
print(Decimal("0.1") + Decimal("0.2")) # 0.3

# 特殊值
float("inf")    # 正无穷
float("-inf")   # 负无穷
float("nan")    # Not a Number

```

complex — 复数（很少语言原生支持）

```

c = 3 + 4j
print(c.real)    # 3.0
print(c.imag)    # 4.0
print(abs(c))    # 5.0（模  $\sqrt{3^2+4^2}$ ）
print(c.conjugate()) # (3-4j)

```

bool — int 的子类

```
# True = 1, False = 0
print(True + True)    # 2
print(False * 10)     # 0

# 关键：隐式布尔转换
# 以下值在条件判断中当作 False：
# None, False, 0, 0.0, "", [], (), {}, set(), range(0)
# 其他值都是 True

# 所以不用写：
if len(items) > 0:
# 直接写：
if items:           # ✓ 更 Pythonic
    process(items)

# 也不要写：
if len(items) == 0:
    print("empty")
# 直接写：
if not items:
    print("empty")
```

None — 空值（类似 null/nil）

```
x = None
print(x is None)    # True ✓ 用 is 比较，不用 ==
print(x == None)    # 也可以但不推荐

# None 不是一个特殊值——它是一个单例对象
# 所有 None 引用指向同一个对象
y = None
print(x is y)       # True
```

4. 场景

场景 1：隐式布尔转换在 API 设计中的影响

```
# 一个函数返回列表——调用方如何判断"没有结果"？
def search_users(query):
    # 如果没找到，返回 [] 还是 None？
    ...
    return results

# 如果返回 []：
users = search_users("nonexistent")
if users:          # [] → False
    process(users)

# 如果返回 None：
users = search_users("nonexistent")
if users is not None and users: # 需要两层判断
    process(users)

# ✓ 最佳实践：函数返回一致的类型结构，让隐式布尔转换帮你
def search_users(query):
    if not query:
        return [] # 返回空列表，而不是 None
    return [user for user in all_users if query in user.name]
```

场景 2：float 精度问题在比较中的陷阱

```
# 写单元测试时容易踩的坑
def test_addition():
    assert 0.1 + 0.2 == 0.3 # ✗ 失败

# ✓ 用 pytest.approx 或 math.isclose
from math import isclose
assert isclose(0.1 + 0.2, 0.3) # ✓
```

```
# 直接用 round
assert round(0.1 + 0.2, 10) == round(0.3, 10) # ✓
```

5. 替代方案对比

类型	Python	其他语言	差异
int	任意精度	通常 32/64 位	Python 不会溢出
float	double	double	都一样有问题
bool	int 子类	独立类型	True+True=2
null	None	null/nil/undefined	单例，用 is 比较
decimal	Decimal	BigDecimal	精度可配置

6. 常见坑

```
# 坑 1: is 和 == 混用
a = 256
b = 256
print(a is b) # True - 小整数缓存 (-5~256)

c = 257
d = 257
print(c is d) # False! 超出缓存范围

# ✓ 判断值用 ==, 判断是不是同一个对象用 is
# None 判断一定要用 is

# 坑 2: 浮点的 NaN 不那么 NaN
nan = float("nan")
print(nan == nan) # False! IEEE 754 规定 NaN ≠ NaN
print(nan is nan) # True (同一个对象)

# 安全判断
import math
```



```
math.isnan(nan) # True ✓

# 坑 3: 空列表不等于 False
print([] == False) # False
print(bool([])) # False
if not []:
    print("empty") # ✓ 正确

# 坑 4: 十进制字符串 vs 数字
# Decimal("0.1") 和 Decimal(0.1) 不一样!
print(Decimal("0.1")) # 0.1
print(Decimal(0.1)) #
0.10000000000000000055511151231257827021181583404541015625
```

7. 代码验证

```
# 验证 int 的任意精度
# 计算 2^100000 最后 10 位
result = pow(2, 100000)
print(str(result)[-10:]) # 瞬间完成, 不会溢出

# 验证 None 是单例
print(None is None) # True
print(id(None)) # 每次运行都一样
```

三、字符串 (str)

1. 是什么

Python 的字符串是 Unicode 字符的不可变序列。它可以是单引号、双引号、三重引号包裹, 功能上完全等价。

2. 解决了什么问题

- 程序要处理文本, 而文本编码和多语言是最容易出错的地方
- Python 3 的 str 原生存 Unicode, 不再有 Python 2 的"str vs unicode"困惑

- f-string 解决了字符串格式化一直以来的痛点：% 格式化可读性差，`.format()` 啰嗦

3. 核心理论

f-string — Python 3.6 最实用的特性

```
name = "Leo"
age = 30
pi = 3.1415926

# 基础插值
print(f"Name: {name}, Age: {age}")

# 任意表达式
print(f"Next year: {age + 1}")
print(f"Pi: {pi:.2f}")          # Pi: 3.14
print(f"Pi: {pi:010.3f}")      # Pi: 000003.142

# 对齐
print(f"|{'left':<10}|")        # |left  |
print(f"|{'right':>10}|")       # |  right|
print(f"|{'center':^10}|")     # | center |

# 百分比
print(f"Rate: {0.856:.1%}")    # Rate: 85.6%

# 逗号分隔
print(f"{1234567:,}")          # 1,234,567

# 日期格式化
from datetime import datetime
now = datetime.now()
print(f"{now:%Y-%m-%d %H:%M}") # 2026-05-23 11:00

# 3.12+ 多行 f-string
data = {"name": "Leo", "score": 95}
print(
    f"Name: {data['name']}\n"
```

```
f"Score: {data['score']}"  
)
```

字符串不可变

```
s = "hello"  
# s[0] = "H" # ✗ TypeError  
s = "H" + s[1:] # ✔ 只能重新创建  
  
# 每次"修改"都创建新字符串  
s = "a"  
s = s + "b" # 创建 "ab" ("a" 还在, 但可能被 GC)
```

join 而不是 +=

```
words = ["hello", "world", "python"]  
  
# ✗  $O(n^2)$  - 每次 + 创建新字符串  
result = ""  
for s in words:  
    result += s + ", "  
  
# ✔  $O(n)$  - 一次性拼接  
result = ", ".join(words) # "hello, world, python"  
  
# join 传入可迭代对象, 比生成器还快  
result = "".join(f(x) for x in large_data)
```

4. 场景

场景 1：大量字符串拼接的性能对比

```
import time  
  
n = 100000  
words = ["word"] * n
```

```

# +=
t0 = time.perf_counter()
result = ""
for w in words:
    result += w
print(f"+=: {time.perf_counter() - t0:.3f}s")

# join
t0 = time.perf_counter()
result = "".join(words)
print(f"join: {time.perf_counter() - t0:.3f}s")

# n = 100,000 时, join 快几十倍

```

场景 2：路径/URL 拼接

```

# 不要用字符串 +
base_url = "https://api.example.com"
endpoint = "/v2/users"
url = base_url + endpoint # 要小心斜杠

# ✔ 用 pathlib 或 urllib.parse
from urllib.parse import urljoin
url = urljoin(base_url, endpoint)

# 文件路径也一样
from pathlib import Path
data_dir = Path("data")
log_file = data_dir / "logs" / "app.log" # ✔ 更清晰的路径拼接

```

5. 替代方案对比

方法	示例	什么时候用
<code>+</code>	<code>"hello " + name</code>	少量拼接
<code>join</code>	<code>", ".join(items)</code>	列表/可迭代对象批量拼接

方法	示例	什么时候用
f-string	<code>f"Name: {name}"</code>	绝大多数场景
<code>.format()</code>	<code>"Name: {}".format(name)</code>	模板需要变量和模板分离时
<code>%</code>	<code>"Name: %s" % name</code>	兼容旧代码或 logging
<code>Template</code>	<code>Template("\$name").substitute(...)</code>	用户提供的模板

建议：日常 90% 用 f-string，批量用 join，模板用 `.format()` 或 `Template`。

6. 常见坑

```
# 坑 1: 字符串切片不报 IndexError
s = "hi"
print(s[0])      # "h"
print(s[5:10])  # "" - 不报错!
print(s[5])      # ✕ IndexError - 但单个索引会报

# 坑 2: str 转 float 可能比较器失败
"1.5" > "10.0"   # True! 字符串比较是字典序, 不是数值
"1.5" > "1.49"   # True - 字典序 5 > 4
# ✔ 比较前先转成数值
float("1.5") > float("10.0") # False

# 坑 3: encode/decode 要指定编码
"中文".encode()      # UTF-8 (默认)
"中文".encode("gbk")  # 如果预期是 GBK
b"\xd6\xd0\xce\xc4".decode("gbk") # "中文"
b"\xe4\xb8\xad\xe6\x96\x87".decode("utf-8") # "中文"

# 坑 4: 单引号和双引号没有区别
'hello' == "hello"   # True
# 只是为了嵌套方便: f"He said: {'hello'}"

# 坑 5: 3.12 之前的 f-string 不能嵌套相同引号
```

```
# ✘ f"Name: {"Leo"}"  
# ✔ f'Name: {"Leo"}'
```

7. 代码验证

```
# 验证 join 的性能优势  
import time  
  
words = ["word"] * 50000  
  
def test_plus():  
    r = ""  
    for w in words:  
        r += w  
    return r  
  
def test_join():  
    return "".join(words)  
  
t1 = time.perf_counter()  
test_plus()  
print(f"+=: {time.perf_counter()-t1:.3f}s")  
  
t1 = time.perf_counter()  
test_join()  
print(f"join: {time.perf_counter()-t1:.3f}s")
```

四、容器类型

1. 是什么

Python 有四种内置容器：list、tuple、set、dict。它们覆盖了"有序/无序"×"可变/不可变"的所有组合。

	有序	无序
可变	list	set, dict(key)
不可变	tuple	frozenset

2. 解决了什么问题

- 每种容器针对不同的使用模式做了优化
- 语法简洁——字面量语法 `[]`, `()`, `{}`, `{k:v}` 比其他语言更干净
- 切片、推导式、解包等操作让数据转换堪称艺术

3. 核心理论

列表（**list**）— 最常用的容器

```
# 创建
nums = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, [1, 2]] # 可以放不同类型

# 索引和切片
nums[0]      # 1
nums[-1]     # 5 (倒数第一个)
nums[1:3]    # [2, 3] (左闭右开)
nums[::2]    # [1, 3, 5] (步进)
nums[::-1]   # [5, 4, 3, 2, 1] (反转)

# 修改
nums.append(6)      # [1, 2, 3, 4, 5, 6]
nums.extend([7, 8]) # [1, 2, 3, 4, 5, 6, 7, 8]
nums.insert(0, 0)   # [0, 1, 2, 3, 4, 5, 6, 7, 8]
nums.pop()          # 删除并返回最后一个
nums.pop(0)          # 删除并返回索引 0
nums.remove(3)       # 删除第一个值为 3 的元素
```

列表推导式 — **Python** 标志性语法：

```

# 基本
squares = [x**2 for x in range(10)]
# 等价于：
squares = []
for x in range(10):
    squares.append(x**2)

# 带条件过滤
evens = [x for x in range(20) if x % 2 == 0]

# 嵌套推导（按 for 的顺序阅读）
matrix = [[i*3 + j for j in range(3)] for i in range(3)]
# [[0,1,2],[3,4,5],[6,7,8]]

# 展开嵌套列表（从里到外读）
flat = [x for row in matrix for x in row]
# [0,1,2,3,4,5,6,7,8]

# 字典列表推导
users = [{"name": n, "score": s} for n, s in zip(names, scores)]

```

元组（**tuple**）— 不可变列表

```

# 创建
point = (3, 4)
single = (1,) # 注意逗号！（1）是 int

# 解包
x, y = point

# 变量交换（底层就是元组打包/解包）
a, b = b, a

# 星号解包
first, *middle, last = [1, 2, 3, 4, 5]
# first=1, middle=[2,3,4], last=5

# 元组可哈希 → 能用做字典键

```



```

d = {(1, 2): "point_a"}

# 命名元组更易读
from collections import namedtuple
Point = namedtuple("Point", ["x", "y"])
p = Point(3, 4)
print(p.x, p.y)  # 3 4

```

集合 (**set**) — 无序不重复

```

s = {1, 2, 3, 3, 2}
print(s)          # {1, 2, 3} (自动去重)

s.add(4)
s.discard(99)     # 删除, 不存在也不报错
s.remove(99)      # ✕ 不存在会报 KeyError

# 集合运算
a = {1, 2, 3}
b = {2, 3, 4}

a & b             # {2, 3}           交集
a | b             # {1, 2, 3, 4}     并集
a - b             # {1}              差集
a ^ b             # {1, 4}           对称差集

# O(1) 查找 vs O(n) 查找
if item in large_set:  # O(1) 无论集合多大
if item in large_list: # O(n) 越慢越慢

```

字典 (**dict**) — 关联数组

```

user = {
    "name": "Leo",
    "age": 30,
    "skills": ["Python", "Go"],
}

```

```

# 安全取值
user.get("name")          # "Leo"
user.get("salary", 0)     # 0 (key 不存在返回默认值)
user["salary"]            # ✕ KeyError

# setdefault – key 不存在才设值
user.setdefault("level", "junior")
user.setdefault("level", "senior") # 不会覆盖

# 更新/合并
d1 = {"a": 1, "b": 2}
d2 = {"b": 3, "c": 4}
d1.update(d2)             # d1 = {"a":1, "b":3, "c":4}
d3 = d1 | d2              # 3.9+ 合并运算符

# 遍历
for key in user:
    print(key, user[key])

for key, value in user.items():
    print(f"{key}={value}")

# 字典推导
squares = {x: x**2 for x in range(5)}
# {0:0, 1:1, 2:4, 3:9, 4:16}

```

4. 场景

场景 1：去重并保留顺序

```

# list → set → list 会丢失顺序
items = [3, 1, 2, 3, 1, 4]
unique = list(set(items)) # [1, 2, 3, 4] – 顺序变了

# ✔ 去重并保留顺序 (利用字典有序了)
unique = list(dict.fromkeys(items)) # [3, 1, 2, 4]

# 或者用循环

```

```
seen = set()
unique = []
for x in items:
    if x not in seen:
        seen.add(x)
        unique.append(x)
```

场景 2 : **zip** 配合 **dict** 构建映射

```
keys = ["name", "age", "city"]
values = ["Leo", 30, "Adelaide"]

# 两个列表变字典
user = dict(zip(keys, values))
# {'name': 'Leo', 'age': 30, 'city': 'Adelaide'}

# 转置字典
original = {"a": 1, "b": 2, "c": 1}
inverted = {}
for k, v in original.items():
    inverted.setdefault(v, []).append(k)
# {1: ['a', 'c'], 2: ['b']}
```

场景 3 : 嵌套列表展开

```
# 任意深度展开
def flatten(nested):
    for item in nested:
        if isinstance(item, (list, tuple)):
            yield from flatten(item)
        else:
            yield item

nested = [1, [2, [3, 4], 5], 6]
list(flatten(nested)) # [1, 2, 3, 4, 5, 6]
```

5. 替代方案对比

场景	方法	效率	可读性
列表 loop 构建	列表推导	✓ 快	✓ 一行
列表 loop 构建	for + append	差不多	啰嗦
字典条件构建	字典推导	✓ 快	✓ 一行
去重	set()	O(n)	✓ 但有损耗
去重且保序	dict.fromkeys	O(n)	推荐
查找存在	in(集合)	O(1)	✓
查找存在	in(列表)	O(n)	✗ 大数据慢

6. 常见坑

```
# 坑 1: 列表的 * 是浅拷贝
matrix = [[0]*3] * 3    # [[0,0,0],[0,0,0],[0,0,0]]
matrix[0][0] = 1
print(matrix)    # [[1,0,0],[1,0,0],[1,0,0]] - 三个子列表是同一个！

# ✓ 正确方式
matrix = [[0]*3 for _ in range(3)]

# 坑 2: 元组的逗号 vs 括号
t = (1)    # int, 不是元组
t = (1,)   # 元组
t = 1,     # 元组 (括号可选)

# 坑 3: 集合的空字面量
s = {}     # 空字典！
s = set()  # 空集合

# 坑 4: dict 的 update 会覆盖
```

```
d = {"a": 1, "b": 2}
d.update({"b": 3, "c": 4})
print(d)  # {'a': 1, 'b': 3, 'c': 4} - b 被覆盖了

# 坑 5: 列表推导的变量泄漏 (Python 2 问题, 3 已修复)
[x for x in range(10)]
# print(x)  # Python 2 会输出 9, Python 3 报 NameError ✓
```

7. 代码验证

```
# 验证 set 的 O(1) 查找
import time

n = 10000000
items_list = list(range(n))
items_set = set(items_list)

target = n - 1

t0 = time.perf_counter()
print(target in items_list)
print(f"list: {time.perf_counter()-t0:.6f}s")

t0 = time.perf_counter()
print(target in items_set)
print(f"set: {time.perf_counter()-t0:.6f}s")

# 百万级数据, list 是微秒级, set 是纳秒级
```

五、控制流

1. 是什么

Python 的控制流除了常见的 `if/for/while` , 还有三个其他语言没有或很晚才有的特性: `for-else`、海象操作符 `:=`、`match-case`。

2. 解决了什么问题

- **for-else** : 解决"是否找到了/是否完成了"这种需要 flag 变量的常见模式
- 海象操作符 : 消除"先赋值再用值"的重复代码
- **match-case** : 替代冗长的 if-elif 链 , 且能匹配数据结构

3. 核心理论

for-else / while-else

`else` 在循环没有被执行 **break** 时触发 :

```
# 找素数
def is_prime(n):
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            print(f"{n} is divisible by {i}")
            break
    else:
        print(f"{n} is prime")
        return True
    return False

# ✓ else 使代码更"扁平"—不需要 maintain 一个 flag 变量

# 实战 : 找可用端口
for port in [8080, 8081, 8082]:
    if check_port(port):
        print(f"Using port {port}")
        break
else:
    raise RuntimeError("No available port")
```

没有 `for-else` 的话 , 需要用 flag :

```
# 没有 for-else
found = False
for port in [8080, 8081, 8082]:
```

```
    if check_port(port):
        print(f"Using port {port}")
        found = True
        break
if not found:
    raise RuntimeError("No available port")
```

海象操作符 := (3.8+)

赋值语句和条件判断合二为一：

```
# 场景 1: if 中使用
# 之前
data = get_data()
if len(data) > 10:
    print(f"Got {len(data)} items")

# 之后
if (n := len(data)) > 10:
    print(f"Got {n} items") # n 在 if 块里也能用

# 场景 2: while 循环
# 之前
line = file.readline()
while line:
    process(line)
    line = file.readline()

# 之后
while line := file.readline():
    process(line)

# 场景 3: 列表推导式过滤
# 筛选变换后非 None 的结果
results = [y for x in data if (y := transform(x)) is not None]

# 场景 4: 正则匹配
```

```
if (match := re.search(pattern, text)):
    print(f"Found: {match.group()}")
```

限制和规范： - 必须用括号括起来才能用在 if 条件中 `if (n := len(x)) > 0:` - 不要滥用——只用在消除重复判断的场景

match-case (3.10+)

Python 的 match-case 不是简单的 switch——它可以模式匹配（匹配结构而不是值）：

```
# 基础—替代 if-elif
def http_status(code):
    match code:
        case 200:
            return "OK"
        case 301 | 302:    # 多个值用 |
            return "Redirect"
        case 404:
            return "Not Found"
        case _:           # 默认
            return "Unknown"

# 进阶—匹配结构
def process_command(cmd):
    match cmd:
        case ("quit",):
            return "bye"

        case ("move", x, y) if 0 <= x <= 100 and 0 <= y <= 100:
            return f"Moving to ({x}, {y})"

        case {"type": "user", "name": name, "age": age}:
            return f"User {name}, {age}"

        case [int() as x, int() as y]:
            return f"Coordinates: {x}, {y}"

        case _:
```



```

        return "Unknown command"

# 匹配枚举
from enum import Enum, auto

class Color(Enum):
    RED = auto()
    GREEN = auto()
    BLUE = auto()

def describe(color):
    match color:
        case Color.RED:
            return "Red like fire"
        case Color.GREEN:
            return "Green like forest"
        case _:
            return "Other color"

```

比 **switch** 强的地方：1. 匹配结构，不仅仅是数值 2. 支持 guard (if 条件) 3. 自动解构赋值 4. `|` 连接多个模式

4. 场景

场景 1：for-else 替代哨兵变量

```

# 查快递状态
tracking_numbers = ["SF123", "SF456", "SF789"]

for tn in tracking_numbers:
    if query_status(tn) == "delivered":
        print(f"Found delivered: {tn}")
        break
else:
    print("No delivered packages found")

```

场景 2：海象在解析中的应用

```
import re

# 多次正则提取
def parse_config(text):
    config = {}
    # 解析 key=value 行
    pattern = re.compile(r"(\w+)\s*=\s*(.*)")

    for line in text.splitlines():
        if m := pattern.match(line.strip()):
            key, value = m.groups()
            config[key] = value
            print(f"  Parsed: {key} = {value}")

    return config
```

场景 3：match-case 的 JSON 路由

```
def handle_event(event):
    """处理各种事件——比 if-elif 干净得多"""
    match event:
        case {"type": "login", "user": user}:
            return handle_login(user)

        case {"type": "logout", "user": user, "session": session}:
            return handle_logout(user, session)

        case {"type": "error", "code": code, "message": msg}:
            return handle_error(code, msg)

        case {"type": "message", "text": text, "reply_to": reply_to}:
            return handle_reply(text, reply_to)

        case {"type": "message", "text": text}:
            return handle_message(text)
```

```
case _:
    raise ValueError(f"Unknown event: {event}")
```

5. 替代方案对比

场景	推荐	不推荐
找东西找到就停	for-else	flag 变量
while 读行	<pre>while line := f.readline():</pre>	<pre>while True: ... if not line: break</pre>
if 中使用赋值 + 判断	<pre>if (n := len(x)) > 0:</pre>	<pre>n = len(x) + if n > 0:</pre>
值匹配	match-case	if-elif-elif...
结构匹配	match-case	isinstance + 手动解包

6. 常见坑

```
# 坑 1: for-else 的 break 判断
def find_item(items, target):
    for i, item in enumerate(items):
        if item == target:
            print(f"Found at {i}")
            break
    else:
        print("Not found")
        return None
    return i

# 注意: 如果是函数中间 return 而不是 break, else 也会执行!
for i in range(3):
    if i == 1:
        return # 从函数返回, 不是 break
else:
    print("This runs!") # ✕ 因为没 break
```

```

# 所以 for-else 只适用 break 场景，不适用 return

# 坑 2：海象操作符的优先级
# ✘ if n := len(x) > 0: # 等价于 n := (len(x) > 0) → n=True/False
if (n := len(x)) > 0: # ✔ 正确

# 坑 3：match-case 的_放在最后
match code:
    case 200: ...
    case _: ...
    case 404: ... # ✘ Unreachable

# 坑 4：match-case 不是 switch—case 后不能跟任意表达式
x = 10
match 1:
    case x: ... # 这不匹配 x 的值！这是赋值给了 x
    # 要用 case _ if x == 1: ... 才行

```

7. 代码验证

```

# 验证海象操作符消除重复
import re
import time

data = "error" * 10000

def without_walrus():
    results = []
    word = ""
    for ch in data:
        if ch == 'e':
            word = "error"
            if len(word) > 0:
                results.append(word)
    return results

def with_walrus():
    results = []
    word = ""

```

```
for ch in data:
    if ch == 'e':
        word = "error"
    if (n := len(word)) > 0:
        results.append((n, word))
return results
```

六、函数

1. 是什么

Python 的函数比其他语言灵活得多——参数系统极其丰富，且函数本身是一等公民（一切皆对象）。

2. 解决了什么问题

- 参数系统的灵活性让 API 设计更优雅——调用方能选择传位置参数还是关键字参数
- `*args` / `**kwargs` 解决了"函数要支持任意数量参数"的问题
- 闭包解决了"函数需要携带上下文"的问题（不用定义一个类）
- 可变默认值陷阱是一个经典测试题——理解它才算理解 Python 对象模型

3. 核心理论

参数系统

```
def complex_func(
    a,                # 位置参数（必填）
    /,                # 分隔符：前面只能位置传参（3.8+）
    b,                # 位置或关键字参数
    *,                # 分隔符：后面只能关键字传参
    c,                # 仅关键字参数
    d=42,             # 默认值参数（可选，在 * 后面也仅限关键字）
    **kwargs          # 关键字参数收集
):
    print(a, b, c, d, kwargs)
```

```

# 正确调用
complex_func(1, 2, c=3, d=4, extra="hello")
# 1 2 3 4 {'extra': 'hello'}

# 错误调用
# complex_func(1, 2, 3, 4, extra="hello") # ✕ c 不能位置传
# complex_func(a=1, b=2, c=3) # ✕ a 不能关键字传

```

为什么需要 / 和 * : - / 前面的参数只能位置传——这是为了以后可以改参数名而不影响已有代码 - * 后面的参数只能关键字传——避免参数太多时搞错顺序

可变参数

```

# *args - 任意数量位置参数
def sum_all(*numbers):
    return sum(numbers)

print(sum_all(1, 2, 3)) # 6
print(sum_all(1, 2, 3, 4, 5)) # 15

# 解包调用
sum_all(*[1, 2, 3]) # 6 - 列表展开传入

# **kwargs - 任意数量关键字参数
def build_url(base, **params):
    query = "&".join(f"{k}={v}" for k, v in params.items())
    return f"{base}?{query}"

print(build_url("http://api.com", name="Leo", age=30))
# http://api.com?name=Leo&age=30

# 解包调用
build_url("http://api.com", **{"name": "Leo", "age": 30})

```

可变默认值陷阱

```
# ❌ 经典大坑
def add_item(item, items=[]):
    items.append(item)
    return items

print(add_item(1))      # [1]
print(add_item(2))      # [1, 2]—不是 [2]！
print(add_item(3))      # [1, 2, 3]

# 原因：默认值在函数定义时求值一次，之后每次都改同一个列表对象
print(add_item.__defaults__) # ([1, 2, 3],)

# ✔ 正确做法
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items

print(add_item(1))      # [1]
print(add_item(2))      # [2]
```

闭包

```
def make_counter(start=0):
    count = start

    def increment(step=1):
        nonlocal count # nonlocal 是关键！不能省
        count += step
        return count

    def reset():
        nonlocal count
        count = start
```

```

    return increment, reset

inc, reset = make_counter(10)
print(inc())      # 11
print(inc(5))     # 16
reset()
print(inc())      # 11

```

闭包的核心：内部函数"记住"了外部函数的变量，即使外部函数已经返回。

lambda

```

# 单行匿名函数
square = lambda x: x ** 2
print(square(5)) # 25

# 常用搭配
pairs = [(1, "b"), (2, "a"), (3, "c")]
pairs.sort(key=lambda x: x[1]) # 按第二个元素排序

sorted([-3, 1, -2, 4], key=lambda x: abs(x))
# [1, -2, -3, 4]

# 限制：只能写表达式，不能写语句
# lambda x: x + 1 # ✔
# lambda x: if x > 0: return x # ✘
# lambda x: x if x > 0 else 0 # ✔ 三元表达式可以

```

4. 场景

场景 1：参数校验 + 关键字参数

```

def create_user(
    /, # username 只能位置传
    username: str,
    *, # 以下必须关键字传
    email: str,
    age: int | None = None,

```



```

    is_admin: bool = False,
):
    """创建用户—明确区分必填和可选"""
    if not username.isalnum():
        raise ValueError("Username must be alphanumeric")

    return {
        "username": username,
        "email": email,
        "age": age,
        "is_admin": is_admin,
    }

# 调用
create_user("Leo", email="leo@example.com")
# create_user(username="Leo", email="...") # ✕ username 不能关键字传

```

场景 2：可变参数装饰器

```

import time
from functools import wraps

def timed(reps=1):
    """测量函数执行时间的装饰器"""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            total = 0
            for _ in range(reps):
                start = time.perf_counter()
                result = func(*args, **kwargs)
                total += time.perf_counter() - start
            print(f"{func.__name__}: avg {total/reps:.6f}s")
            return result
        return wrapper
    return decorator

@timed(reps=10)
def slow_function():

```

```
time.sleep(0.01)

slow_function() # slow_function: avg 0.0100xxxxs
```

5. 替代方案对比

需求	Python	其他语言	说明
可选参数	默认值	重载	Python 简洁
任意数量参数	*args	可变参数/C++initializer_list	差不多
关键字参数	**kwargs	无/命名参数	Python 独家
仅位置参数	/	传统	3.8+
仅关键字参数	*	无	Python 独家
匿名函数	lambda	箭头函数	其他语言广泛

6. 常见坑

```
# 坑 1: lambda 闭包延迟绑定
funcs = [lambda: i for i in range(5)]
print([f() for f in funcs]) # [4, 4, 4, 4, 4] !不是 [0,1,2,3,4]

# 原因: lambda 中的 i 是引用, 循环结束时 i=4
# ✔ 默认参数绑定当前值
funcs = [lambda i=i: i for i in range(5)]
print([f() for f in funcs]) # [0, 1, 2, 3, 4]

# 坑 2: 默认值引用同一个可变对象
# 参见上面的可变默认值陷阱

# 坑 3: *args 和 **kwargs 混合时**kwargs 必须在最后
# def f(**kwargs, *args): # ✕ SyntaxError

# 坑 4: nonlocal 只向上查找最近的非全局作用域
```

```

x = "global"

def outer():
    x = "outer"
    def inner():
        x = "inner"
        def innermost():
            nonlocal x  # 这是 inner 的 x, 不是 outer 的
            x = "changed"
        innermost()
    print(x)  # "inner"—outer 的 x 没变

```

7. 代码验证

```

# 验证默认值在定义时创建
import time

def get_default_list(value):
    """返回新列表"""
    return [value]

def get_default_list_bad(
    value,
    items=[time.time()],  # 定义时创建, 后续调用都用同一个
):
    items.append(value)
    return items

print(get_default_list_bad(1))  # [时间戳, 1]
print(get_default_list_bad(2))  # [时间戳, 1, 2]

# 验证函数对象的 __defaults__
print(get_default_list_bad.__defaults__)
# 显示同一个列表对象

```

```

# 验证 nonlocal 作用域链
def make_counters():
    counters = {"a": 0, "b": 0}

```

```
def inc_a():
    counters["a"] += 1 # 不需要 nonlocal!
    return counters["a"]

def inc_b():
    counters["b"] += 1
    return counters["b"]

# 为什么这里不用 nonlocal?
# 因为 counters 是 dict——我们修改的是内容，不是重新赋值
# nonlocal 只在重新绑定变量名时（=赋值）需要

return inc_a, inc_b

inc_a, inc_b = make_counters()
print(inc_a()) # 1
print(inc_a()) # 2
print(inc_b()) # 1
```

七、异常处理

1. 是什么

Python 的异常处理采用 **EAFP** (Easier to Ask Forgiveness than Permission) 哲学——先试着做，出错了再处理。这和 Java/C 的 **LBYL** (Look Before You Leap) 形成对比。

2. 解决了什么问题

```
# LBYL (其他语言风格)
def divide_list(values, divisor):
    if divisor == 0:
        return None
    results = []
    for v in values:
```

```

        if isinstance(v, (int, float)):
            results.append(v / divisor)
        else:
            results.append(None)
    return results

# EAFP (Python 风格)
def divide_list(values, divisor):
    results = []
    for v in values:
        try:
            results.append(v / divisor)
        except (ZeroDivisionError, TypeError):
            results.append(None)
    return results

```

EAFP 的优势：- 代码更短——不用重复检查条件 - 没有竞态条件——检查和操作之间不会有其他线程改变状态 - 异常路径和正常路径分离

3. 核心理论

```

# 完整结构
try:
    result = risky_operation()
except ValueError as e:          # 捕获特定异常
    print(f"Value error: {e}")
except (IOError, OSError) as e:  # 捕获多个异常
    print(f"IO error: {e}")
except Exception:                # 捕获所有（少用）
    print("Something wrong")
    raise                        # 重新抛出
else:
    print(f"Success: {result}")  # 没异常才执行
finally:
    cleanup()                   # 无论是否有异常都执行

```

else 的妙用：把"成功处理逻辑"放在 **else** 中，而不是在 **try** 块末尾——这样不会意外捕获到成功处理逻辑中的异常。

自定义异常

```
class AppError(Exception):
    """应用异常基类"""
    pass

class ValidationError(AppError):
    def __init__(self, field: str, message: str):
        self.field = field
        self.message = message
        super().__init__(f"{field}: {message}")

class NotFoundError(AppError):
    def __init__(self, resource: str, id: int):
        self.resource = resource
        self.id = id
        super().__init__(f"{resource} #{id} not found")

# 使用
raise ValidationError("email", "Invalid format")
raise NotFoundError("User", 42)
```

异常链

```
def load_config(path):
    try:
        with open(path) as f:
            return json.load(f)
    except FileNotFoundError as e:
        raise AppError(f"Config not found: {path}") from e
    except json.JSONDecodeError as e:
        raise AppError(f"Invalid JSON in {path}") from e

# "from e" 保留了原始异常信息
try:
    load_config("missing.json")
except AppError:
    import traceback
```

```
traceback.print_exc()

# 显示完整的异常链：FileNotFoundError → AppError
```

4. 场景

场景 1：异常嵌套的层级

```
# 多层函数调用时，在最上层统一处理
class ServiceError(Exception):
    pass

def db_query(sql):
    # 最底层（原始异常）
    raise ConnectionError("DB connection refused")

def get_user(user_id):
    try:
        return db_query(f"SELECT * FROM users WHERE id={user_id}")
    except ConnectionError as e:
        raise ServiceError("Database unavailable") from e

def handler():
    # 最上层（统一处理）
    try:
        user = get_user(42)
        return {"status": "ok", "data": user}
    except ServiceError as e:
        return {"status": "error", "message": str(e)}
```

场景 2：with contextlib.suppress 优雅忽略

```
from contextlib import suppress

# 不用 suppress：
try:
    os.remove("temp.txt")
except FileNotFoundError:
    pass
```

```

# 用 suppress (更简洁) :
with suppress(FileNotFoundError):
    os.remove("temp.txt")

# 多个异常
with suppress(FileNotFoundError, PermissionError):
    os.remove("protected.txt")

```

5. 替代方案对比

模式	Python 推荐	适用场景
先检查再做	LBYL	简单条件判断，内部操作
先试后处理	EAFP	IO、网络、用户输入
忽略特定异常	suppress	清理操作（删除不存在的文件）
重新抛出	<code>raise</code>	需要记录日志但不想处理
异常链	<code>raise ... from e</code>	转换异常种类时

6. 常见坑

```

# 坑 1: except 的顺序
try:
    process()
except Exception:      # 先捕获所有
    print("caught")
except ValueError:     # 永远不会到这里
    print("never")

# ✔ 先具体后通用
try:
    process()
except ValueError:
    print("Value error")

```



```

except TypeError:
    print("Type error")
except Exception:
    print("Other error")

# 坑 2: except: 没指定会捕获 SystemExit 和 KeyboardInterrupt
# 永远不要裸 except:
try:
    user_input()
except:      # 会吞掉 Ctrl+C !
    pass

# ✔ 至少写 except Exception:
try:
    user_input()
except Exception:
    pass      # Ctrl+C 仍然工作

# 坑 3: finally 的 return 会覆盖其他 return
def f():
    try:
        return "try"
    finally:
        return "finally" # ✗ 覆盖了 try 的 return

print(f()) # "finally"

# 坑 4: 空 except 不推荐——你永远不知道什么异常

```

7. 代码验证

```

# 验证 else 的用途
try:
    result = int("42")
except ValueError:
    print("Invalid number")
else:
    # 只有成功才执行

```

```
print(f"Result is {result}")  
# 如果这里写在 try 里不小心抛异常，也会被上面的 except 捕获
```

八、面向对象（**OOP**）基础

1. 是什么

面向对象编程（OOP）是一种将数据和操作数据的方法组织成"对象"，每个对象包含数据（属性）和方法（行为）。

Python 的 OOP 比大多数语言更灵活——它支持多重继承、方法重写、鸭子类型，以及一切皆对象的哲学。

基础语法速览：

```
class Dog:  
    species = "Canis familiaris"  # 类属性（所有实例共享）  
  
    def __init__(self, name, age):  
        self.name = name  # 实例属性  
        self.age = age  
  
    def bark(self):  
        return f"{self.name} says woof!"  
  
    def __str__(self):  
        return f"{self.name} ({self.age}yo)"  
  
buddy = Dog("Buddy", 3)  
print(buddy.bark())      # "Buddy says woof!"  
print(buddy.species)     # "Canis familiaris"  
print(buddy)             # "Buddy (3yo)"
```

关键点：- `self` 是实例本身的引用（类似其他语言的 `this`），必须作为第一个参数 -
`__init__` 是构造方法（初始化实例），不是构造函数（对象已经创建了再初始化） -

`__str__` 是魔术方法，控制 `print(obj)` 的输出 - 实例方法、类方法、静态方法有不同的第一个参数约定

2. 解决了什么问题

问题 1：数据和操作分离

没有 OOP 时，操作数据的函数和数据是分开的：

```
# 函数式风格
def create_dog(name, age):
    return {"name": name, "age": age}

def bark(dog):
    return f"{dog['name']} says woof!"

def celebrate_birthday(dog):
    dog['age'] += 1
```

每增加一个操作，就要加一个新函数。谁创建了数据谁负责操作——没有自然的分组。

OOP 把数据和操作封装在一起——"狗的年龄增加"不再是外部函数的职责，而是 Dog 自己的方法。

问题 2：代码复用

直接复制粘贴来修改功能会导致大量重复代码。OOP 的继承允许你在一个地方修改，所有子类自动获得。

问题 3：现实世界建模

OOP 的"对象"和现实世界的"事物"有自然对应关系：用户、订单、商品、请求.....这让代码结构更容易理解。

3. 核心理论

3.1 类属性 vs 实例属性

```
class Employee:
    # 类属性——所有实例共享
    company = "Acme Inc"
    raise_rate = 1.05

    def __init__(self, name, salary):
        # 实例属性——每个实例独有
        self.name = name
        self.salary = salary

    def apply_raise(self):
        self.salary = int(self.salary * self.raise_rate)

e1 = Employee("Alice", 50000)
e2 = Employee("Bob", 60000)

print(e1.company) # "Acme Inc"
print(e2.company) # "Acme Inc"

# 修改类属性影响所有实例
Employee.raise_rate = 1.10
e1.apply_raise()
print(e1.salary) # 55000

# 修改实例属性不影响类
Employee.company = "New Corp"
print(e1.company) # "New Corp"
print(e2.company) # "New Corp"

# 但给实例设置同名属性会屏蔽类属性
e1.company = "Private Co"
print(e1.company) # "Private Co" — 实例属性覆盖
print(e2.company) # "New Corp" — 还是类属性
```

属性查找顺序：实例属性 → 类属性 → 父类（MRO 顺序）

3.2 实例方法、类方法、静态方法

```
class Date:
    def __init__(self, year, month, day):
        self.year = year
        self.month = month
        self.day = day

    # 实例方法—需要访问实例
    def format(self):
        return f"{self.year}-{self.month:02d}-{self.day:02d}"

    # 类方法—需要访问类（第一个参数是 cls，不是 self）
    @classmethod
    def from_string(cls, date_str):
        """替代构造器：由字符串创建实例"""
        year, month, day = map(int, date_str.split("-"))
        return cls(year, month, day)

    # 静态方法—不需要访问类或实例（就是个普通函数放在类里）
    @staticmethod
    def is_valid(date_str):
        """验证日期字符串格式"""
        try:
            parts = date_str.split("-")
            return len(parts) == 3
        except Exception:
            return False

# 实例方法：通过实例调用
d = Date(2026, 5, 23)
print(d.format())  # "2026-05-23"

# 类方法：通过类或实例调用
d2 = Date.from_string("2026-06-01")
print(d2.format())  # "2026-06-01"
```

```
# 静态方法：通过类或实例调用
print(Date.is_valid("2026-05-23")) # True
```

方法类型	第一个参数	能访问实例	能访问类	什么时候用
实例方法	<code>self</code>	✓	✓	绝大多数方法
类方法	<code>cls</code>	✗	✓	工厂方法、替代构造器
静态方法	无	✗	✗	工具函数，语义上属于这个类

3.3 继承

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        raise NotImplementedError("Subclasses must implement")

    def __str__(self):
        return f"Animal: {self.name}"

class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"

    def fetch(self):
        return f"{self.name} fetches the ball"

class Cat(Animal):
    def __init__(self, name, lives=9):
        super().__init__(name) # 调用父类构造
        self.lives = lives

    def speak(self):
```

```

        return f"{self.name} says Meow!"

    def __str__(self):
        return f"Cat: {self.name} ({self.lives} lives left)"

class RobotDog(Dog):
    """多重继承层次——覆盖和不覆盖"""
    def speak(self):
        return f"{self.name} says Beep beep!"

animals = [Dog("Buddy"), Cat("Whiskers"), RobotDog("R2D2")]
for a in animals:
    print(a.speak()) # 多态：不同类，同一接口
# Buddy says Woof!
# Whiskers says Meow!
# R2D2 says Beep beep!

print(isinstance(animals[0], Animal)) # True
print(isinstance(animals[2], Dog))    # True (继承链)
print(issubclass(RobotDog, Animal))   # True

```

super() 的作用：调用父类方法。在 `__init__` 中调用 `super().__init__()` 是最常见的模式——确保父类的初始化逻辑被执行。

3.4 魔术方法 (**Magic Methods / Dunder Methods**)

Python 的类可以通过实现特定名称的方法来参与语言的内建操作：

```

class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        """开发者看到的表示"""
        return f"Vector({self.x}, {self.y})"

```

```

def __str__(self):
    """用户看到的表示"""
    return f"({self.x}, {self.y})"

def __add__(self, other):
    """支持 + 运算符"""
    return Vector(self.x + other.x, self.y + other.y)

def __sub__(self, other):
    """支持 - 运算符"""
    return Vector(self.x - other.x, self.y - other.y)

def __eq__(self, other):
    """支持 == 比较"""
    return self.x == other.x and self.y == other.y

def __abs__(self):
    """支持 abs()"""
    return (self.x**2 + self.y**2) ** 0.5

def __bool__(self):
    """支持布尔判断"""
    return self.x != 0 or self.y != 0

v1 = Vector(3, 4)
v2 = Vector(1, 2)

print(v1)                # (3, 4) ← __str__
print(repr(v1))           # Vector(3, 4) ← __repr__
print(v1 + v2)            # (4, 6) ← __add__
print(v1 - v2)            # (2, 2) ← __sub__
print(v1 == Vector(3, 4)) # True ← __eq__
print(abs(v1))            # 5.0 ← __abs__
print(bool(Vector(0, 0))) # False ← __bool__

```

常用魔术方法速查：

类别	方法	触发时机
构造	<code>__init__</code> , <code>__new__</code>	创建对象
字符串	<code>__str__</code> , <code>__repr__</code> , <code>__format__</code>	print, str, f-string
算术	<code>__add__</code> , <code>__sub__</code> , <code>__mul__</code> , <code>__truediv__</code>	+, -, *, /
比较	<code>__eq__</code> , <code>__lt__</code> , <code>__gt__</code> , <code>__le__</code> , <code>__ge__</code>	==, <, >, <=, >=
容器	<code>__len__</code> , <code>__getitem__</code> , <code>__setitem__</code> , <code>__contains__</code>	len, [], in
迭代	<code>__iter__</code> , <code>__next__</code>	for 循环
可调 用	<code>__call__</code>	obj()
上下 文	<code>__enter__</code> , <code>__exit__</code>	with 语句

3.5 property — 控制属性访问

```
class Temperature:
    def __init__(self, celsius=0):
        self._celsius = celsius  # 约定：_ 前缀表示"内部使用"

    @property
    def celsius(self):
        """只读属性（但没有 setter 就无法赋值）"""
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        """setter：赋值时做校验"""
        if value < -273.15:
```

```

        raise ValueError("Temperature below absolute zero!")
    self._celsius = value

    @property
    def fahrenheit(self):
        """计算属性（只读）"""
        return self._celsius * 9 / 5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        self._celsius = (value - 32) * 5 / 9

# 使用
t = Temperature(25)
print(t.celsius)                # 25
t.celsius = 30                  # ✓ 调用 setter
print(t.fahrenheit)             # 86.0
# t.celsius = -300              # ✗ ValueError
t.fahrenheit = 100              # ✓ 设置 fahrenheit 连带着改 celsius
print(t.celsius)                # 37.78

```

property 的价值：让你先写简单的属性访问，后续再添加校验/计算逻辑而不改变调用方的代码。

4. 场景

场景 1：数据验证类

```

class User:
    """带验证的用户类"""
    def __init__(self, username, email, age=None):
        self.username = username    # 触发 setter
        self.email = email
        self.age = age

    @property
    def username(self):
        return self._username

```

```

    @username.setter
    def username(self, value):
        if not isinstance(value, str) or len(value) < 3:
            raise ValueError("Username must be at least 3 characters")
        if not value.isalnum():
            raise ValueError("Username must be alphanumeric")
        self._username = value

    @property
    def email(self):
        return self._email

    @email.setter
    def email(self, value):
        if "@" not in value:
            raise ValueError("Invalid email address")
        self._email = value

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value is not None and (value < 0 or value > 150):
            raise ValueError("Age must be between 0 and 150")
        self._age = value

# 使用
u = User("Leo", "leo@example.com", 30) # ✓
# u2 = User("L", "leo@.com")           # ✗ ValueError

```

场景 2：链式 API（类似 **pandas** 风格）

```

class Query:
    def __init__(self, data):
        self._data = list(data)

```

```

def filter(self, predicate):
    self._data = [x for x in self._data if predicate(x)]
    return self # 返回 self 实现链式调用

def map(self, func):
    self._data = [func(x) for x in self._data]
    return self

def sort(self, key=None, reverse=False):
    self._data = sorted(self._data, key=key, reverse=reverse)
    return self

def limit(self, n):
    self._data = self._data[:n]
    return self

def to_list(self):
    return self._data

# 链式使用
result = (
    Query([1, 2, 3, 4, 5, 6])
    .filter(lambda x: x > 2)
    .map(lambda x: x ** 2)
    .sort(reverse=True)
    .limit(2)
    .to_list()
)
print(result) # [36, 25]

```

5. 替代方案对比

场景	OOP	替代方案
简单数据容器	class + init	dataclass / namedtuple
带验证的属性	@property	手动 getter/setter

场景	OOP	替代方案
单方法行为	class + 方法	闭包 (closure)
多种类型行为	继承 + 多态	singledispatch
简单命名元组	namedtuple / dataclass	class 太啰嗦

什么时候用 **class** , 什么时候不用 : - 有内部状态需要维护 → class - 只有行为没有状态 → 用函数 - 只有数据没有行为 → 用 dataclass / namedtuple - 需要多种实现的同一接口 → 用 ABC / Protocol

6. 常见坑

```
# 坑 1: 可变默认值 (跟函数一样的问题!)
class Bad:
    def __init__(self, items=[]): # ✕ 所有实例共享同一个列表
        self.items = items

a = Bad()
b = Bad()
a.items.append(1)
print(b.items) # [1] - 被 a 污染了

class Good:
    def __init__(self, items=None):
        self.items = items if items is not None else []

# 坑 2: 双下划线属性不是私有——是名称修饰
class Secret:
    def __init__(self):
        self.__secret = 42

s = Secret()
# print(s.__secret) # ✕ AttributeError
print(s._Secret__secret) # 42 - 仍然能访问!
# Python 没有真正的私有属性, __ 只是避免子类意外覆盖
# 单下划线 _ 是约定——"这是内部实现, 不要碰"
```

```

# 坑 3: 继承时忘记调用 super().__init__()
class Parent:
    def __init__(self):
        self.initialized = True

class Child(Parent):
    def __init__(self):
        pass # 忘记调用 super().__init__()

c = Child()
# print(c.initialized) # ✕ AttributeError – Parent.__init__ 没执行

# 坑 4: @staticmethod 和 @classmethod 的误用
# 如果方法里用到了 self, 永远不要用 @staticmethod
class MathUtils:
    @staticmethod
    def add(a, b): # ✓ 不需要访问类或实例—纯工具
        return a + b

    @classmethod
    def from_polar(cls, r, theta): # ✓ 需要返回 cls 实例
        return cls(r * cos(theta), r * sin(theta))

# 坑 5: 多重继承的 MRO (方法解析顺序)
class A:
    def method(self):
        return "A"

class B(A):
    def method(self):
        return "B"

class C(A):
    def method(self):
        return "C"

class D(B, C):
    pass

```

```
d = D()
print(d.method()) # "B" - C3 线性化算法决定
print(D.__mro__)  # D → B → C → A → object
```

7. 代码验证

```
# 验证 isinstance 和 type 的区别
class Parent:
    pass

class Child(Parent):
    pass

c = Child()

print(type(c) is Child) # True
print(type(c) is Parent) # False - type 精确匹配

print(isinstance(c, Child)) # True
print(isinstance(c, Parent)) # True - isinstance 走继承链
```

```
# 验证 property 的延迟计算
class Lazy:
    @property
    def expensive(self):
        """每次访问都重新计算"""
        import time
        time.sleep(1)
        print("Computing...")
        return 42

    @cached_property
    def cached(self):
        """只计算一次，之后缓存"""
        import time
        time.sleep(1)
        print("Caching...")
        return 99
```

```
from functools import cached_property
l = Lazy()
print(l.cached) # 第一次：计算
print(l.cached) # 第二次：直接返回缓存值
```

九、文件 I/O 与 `pathlib`

1. 是什么

Python 处理文件有两种主要方式：传统的 `open()` + `with` 语句，以及现代的 `pathlib.Path` 对象（Python 3.4+）。

2. 解决了什么问题

- 传统文件操作需要 `open/read/write/close` 手动管理生命期
- `os.path` 的函数式 API（`os.path.join`，`os.path.exists`）不 OOP，难链式调用
- 跨平台路径分隔符（`/` vs `\`）需要手动处理
- `pathlib` 用对象方法替代了零散的 `os.path` 函数

3. 核心理论

with 语句（上下文管理器）

```
# 不需要手动 close——with 退出时自动关闭
with open("data.txt", "r", encoding="utf-8") as f:
    content = f.read() # 整个文件
    lines = f.readlines() # 按行列表

# 大文件逐行读取（不加载全部到内存）
with open("large.log", "r") as f:
    for line in f: # 惰性读取
        process(line)
```



```
# 写文件
with open("output.txt", "w", encoding="utf-8") as f:
    f.write("Hello, World!\n")
    f.writelines(["line1\n", "line2\n"])
```

打开模式速查：

模式	读/写	指针位置	文件存在	文件不存在
"r"	读	开头	✓	✗ 报错
"r+"	读写	开头	✓	✗ 报错
"w"	写	开头	✗ 清空	✓ 创建
"w+"	读写	开头	✗ 清空	✓ 创建
"a"	写	末尾	✓ 追加	✓ 创建
"a+"	读写	末尾	✓ 追加	✓ 创建
"x"	写	开头	✗ 报错	✓ 创建（排他）

pathlib（推荐）

```
from pathlib import Path

# 路径操作
p = Path("data/config.json")
p.parent          # data/
p.stem            # config
p.suffix          # .json
p.name            # config.json

# 检查
p.exists()        # True/False
p.is_file()
p.is_dir()
```

```

# 读写文本
p.read_text()          # 整个文件为字符串
p.write_text("hello")  # 写入字符串

# 读写字节
p.read_bytes()         # bytes
p.write_bytes(b"\x00\x01") # 写入

# 遍历
for py_file in Path("src").rglob("*.py"):
    print(py_file)

# 创建目录
Path("data/logs").mkdir(parents=True, exist_ok=True)

# 重命名/移动
Path("old.txt").rename("new.txt")

# 路径拼接（重载了 / 运算符）
data_dir = Path("data")
log = data_dir / "logs" / "app.log"

```

4. 场景

场景 1：递归查找并处理文件

```

from pathlib import Path

def find_and_process(root, pattern, callback):
    """在 root 下递归查找匹配 pattern 的文件，对每个文件调用 callback"""
    root = Path(root)
    for filepath in root.rglob(pattern):
        if filepath.is_file():
            callback(filepath)

# 统计所有 .py 文件的行数
total_lines = 0
def count_lines(path):

```

```

    global total_lines
    content = path.read_text()
    lines = content.count("\n") + 1
    total_lines += lines
    print(f"{path}: {lines} lines")

find_and_process(".", "*.py", count_lines)
print(f"Total: {total_lines} lines")

```

场景 2：安全写入（防止写一半崩溃）

```

from pathlib import Path
import tempfile
import os

def safe_write(path: str, content: str):
    """先写到临时文件，再原子替换"""
    path = Path(path)
    # 在同一个目录下创建临时文件（保证跨设备）
    with tempfile.NamedTemporaryFile(
        mode="w",
        dir=path.parent,
        suffix=".tmp",
        delete=False,
    ) as tmp:
        tmp.write(content)
        tmp_path = tmp.name

    # 原子替换
    os.replace(tmp_path, path)

# 使用
safe_write("config.json", '{"version": 2}')

```

场景 3：内存文件（StringIO / BytesIO）

```

from io import StringIO, BytesIO

```

```
# 字符串当作文件操作（测试时超有用）
buffer = StringIO()
buffer.write("Hello\n")
buffer.write("World\n")

buffer.seek(0)
print(buffer.read()) # "Hello\nWorld\n"

# 字节缓冲
bio = BytesIO()
bio.write(b"\x00\x01\x02")
bio.seek(0)
data = bio.read()
```

5. 替代方案对比

操作	旧方式（ os.path ）	新方式（ pathlib ）
路径拼接	<code>os.path.join("a", "b")</code>	<code>Path("a") / "b"</code>
文件名	<code>os.path.basename(p)</code>	<code>Path(p).name</code>
扩展名	<code>os.path.splitext(p)[1]</code>	<code>Path(p).suffix</code>
文件存在	<code>os.path.exists(p)</code>	<code>Path(p).exists()</code>
创建目录	<code>os.makedirs(p, exist_ok=True)</code>	<code>Path(p).mkdir(parents=True)</code>
遍历	<code>os.walk(root)</code>	<code>Path(root).rglob("*")</code>

推荐：新代码全部用 `pathlib`。

6. 常见坑

```
# 坑 1：文件打开后忘记 close
f = open("data.txt")
data = f.read()
# 没有 close！如果在 with 块中抛出异常，文件可能不会关闭
```

```

# 当然用 with 就不会有这个问题
with open("data.txt") as f:
    data = f.read()
# 自动关闭

# 坑 2: pathlib 的 / 运算符返回 Path, 不是 str
p = Path("data") / "file.json"
# 需要 str 的地方要显式转换
# json.load(open(p))      # ✔ 有些函数接受 Path
# os.system(f"cat {p}")    # ✗ TypeError
os.system(f"cat {str(p)}") # ✔

# 坑 3: 文本模式 vs 二进制模式
# Windows 上 "r" 模式会转换 \r\n → \n
# 处理图片/压缩包要用二进制模式
with open("image.jpg", "rb") as f:
    data = f.read() # bytes, 不会被转换

```

7. 代码验证

```

# 验证 pathlib / 运算符是纯路径操作 (不访问磁盘)
from pathlib import Path

p = Path("/nonexistent/directory") / "file.txt"
print(p) # /nonexistent/directory/file.txt
# 不会报错——路径只是字符串操作

# verify 存在检查才访问磁盘
print(p.exists()) # False — 这次才真的检查磁盘

```

十、模块与 import

1. 是什么

Python 的模块系统让代码可以按文件组织，并通过 `import` 跨文件复用。每个 `.py` 文件就是一个模块。

2. 解决了什么问题

- 代码拆分到不同文件，避免一个文件几千行
- 命名空间隔离——`module.function()` 不会冲突
- 复用已有的库——标准库和第三方包

3. 核心理论

```
# 三种 import 方式
import os                # os 命名空间可用, 用 os.path.join()
from pathlib import Path  # Path 直接可用
from datetime import datetime as dt # 别名

# 绝对 vs 相对导入
# ✓ 推荐绝对导入
from my_package.utils.helpers import parse_date

# ✗ 相对导入容易混淆
# from ..utils.helpers import parse_date

# 模块也是对象
import math
print(math.__name__) # "math"
print(math.pi)      # 3.14159...
```

if name == "main"

```
# 当文件被直接运行时 __name__ = "__main__"
# 当文件被 import 时 __name__ = 模块名
def main():
    print("Running directly")

if __name__ == "__main__":
    main()
```

这个模式解决了"可执行/可导入"的双重需求：- `python script.py` → 执行 `main()` - `import script` → 不执行 `main()`

模块搜索路径

```
import sys
print(sys.path)
# ['', '/usr/lib/python3.x', '/usr/local/lib/python3.x/site-packages', ...]

# 可以动态添加
sys.path.append("/my/custom/path")

# 但推荐用 PYTHONPATH 环境变量或安装到 site-packages
```

4. 场景

场景 1：项目的标准包结构

```
myproject/
├─ pyproject.toml      # 项目元数据和依赖
├─ src/
│   └─ myproject/
│       ├── __init__.py  # 包初始化
│       ├── main.py      # 入口
│       ├── config.py    # 配置
│       └─ utils/
│           ├── __init__.py
│           └─ helpers.py
└─ tests/
    ├── test_main.py
    └─ test_helpers.py
```

```
# src/myproject/main.py
from myproject.config import load_config
from myproject.utils.helpers import format_date

def main():
    config = load_config()
    print(format_date(config["start_date"]))
```

```
if __name__ == "__main__":  
    main()
```

5. 常见坑

```
# 坑 1: 循环导入  
# a.py: from b import foo  
# b.py: from a import bar  
# 运行时会报 ImportError  
  
# 解决方案:  
# 1. 把公用的放第三个文件  
# 2. 延迟导入 (在函数内 import)  
# 3. 重构结构消除循环依赖  
  
# 坑 2: import * 污染命名空间  
from os import * # ✖ 不推荐  
# 你不知道导入了什么, 可能会覆盖已有变量  
  
from pathlib import Path # ✔ 明确  
  
# 坑 3: __init__.py 为空时, import package 不会自动导入子模块  
# 需要在 __init__.py 显式 import  
# __init__.py  
# from . import config  
# from . import utils  
  
# 坑 4: 相对导入在 __main__ 中不能用  
# 如果你的文件是入口 (__main__), 里面不能用 from . import xxx
```

6. 代码验证

```
# 验证 __name__ 的不同值  
# running_as_main.py  
print(f"__name__ = {__name__}") # 直接运行 → "__main__"
```

总结

知识点	一句话记法	核心价值
Python 哲学	显式优于隐式，可读性优先	知道为什么 Python 这样设计
数据类型	int 无限、bool 是 int 子类、None 判断用 is	避开常见类型坑
字符串	用 f-string 格式化，用 join 拼接	性能和可读性双赢
容器	list 最常用、set 去重、dict 关联	选择正确的容器
控制流	for-else 替代哨兵、:= 消除重复、match 替代 if-elif	少写样板代码
函数	参数系统灵活、默认值陷阱要避免	API 设计的核心
OOP	class 封装数据和行为、property 控制访问	数据和操作在一起
异常	EAFP、异常链保留上下文	生产级错误处理
文件 I/O	用 with、用 pathlib	更安全更现代
模块	<code>__name__ == "__main__"</code> 区分入口和库	代码组织的基础

明天预告：**Day 2** — 进阶语法 - 迭代器（Iterable vs Iterator、自定义迭代器） - 闭包（Closure、nonlocal） - 装饰器（@语法糖、带参数装饰器、functools.wraps） - 生成器（yield、yield from、生成器表达式） - 上下文管理器（with、contextmanager、**enter/exit**）